

Lesson Notes

MODULE 3 | LESSON 1

DEEPPVAR: VALUE AT RISK MEETS LONG-SHORT TERM MEMORY

Reading Time	40 minutes
Prior Knowledge	Value at Risk (VaR), Long-Short Term Memory (LSTM), Monte Carlo simulation
Keywords	VaR Backtests, Probabilistic forecasting

In the previous module, we investigated some of the interesting features of variance in financial returns, e.g., how it relates risk-neutral pricing of options and how it can include jumps in addition to a continuous term. Variance plays an important role in risk estimation, especially many value at risk models. In this module, we deploy state-of-the-art deep learning to the challenge of VaR forecasting. Along the way, we learn how to assess a VaR model.

1. From Recurrent Neural Networks to Long-Short Term Memory

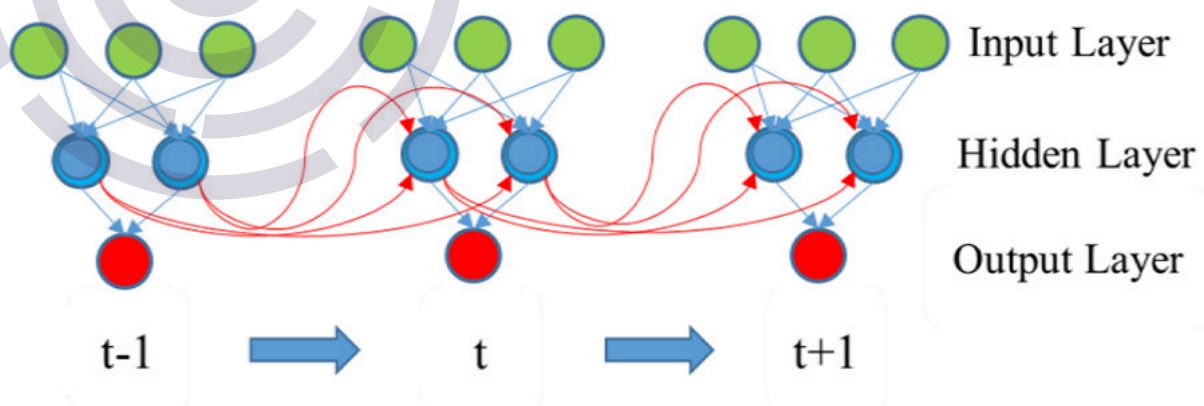
Our focus in this course is risk management. Given that this course is near the end of the program, we can assume your familiarity with many crucial concepts, especially related to machine learning (and deep learning in particular) but also stochastic methods and modeling, and financial econometrics. This is not to say that you are expected to remember every detail and formula from those courses, though of course there may be times you need to review some of that material. All of this is just to say that, given the focus of this course, we will not delve too deeply into the supporting material that was already covered; instead, we will build on and make use of that material toward understanding and applying risk management tools and techniques. With that said, we will continue to view and analyze Python implementations of relevant techniques and concepts when this can be done to illustrate the risk management thrust of our module objectives.

With that said, let's build up to a basic overview of the LSTM framework, since it is a building block in the DeepVaR neural network and we return to it in lessons 3 and 4 of this module.

1.1 Recurrent Neural Network (RNN)

First, recall the basic architecture of the recurrent neural network (RNN), as seen in figure 1.

Figure 1: Unfolded RNN with Single Hidden Layer



Source: Xie, Chenlei et al. "A Long Short-Term Memory Neural Network Model for Knee Joint Acceleration Estimation Using Mechanomyography Signals." *International Journal of Advanced Robotic Systems*, vol. 17, no. 6, 2020, pp. 1–10. <https://journals.sagepub.com/doi/10.1177/1729881420968702>

We see there are three inputs to the neural network, represented by the input layer. In this very simplistic version of the RNN, there is but one hidden layer consisting of only two hidden units. We call this RNN diagram "unfolded" because it only consists of three cells: one output cell and two hidden layer cells. (One could argue it consists of six cells in total if you consider the input layer as consisting of legitimate cells.) In the diagram, we see nine blue and red cells because the same three cells of the actual RNN are repeated for each time step.

Each hidden blue cell has three sets of weights to train, which impact its output:

- one set of weights leading to itself for use in the next time step,
- one leading to the other hidden cell for use in the next time step, and
- one that leads to the output cell.

In a similar fashion, each blue cell takes input from five cells:

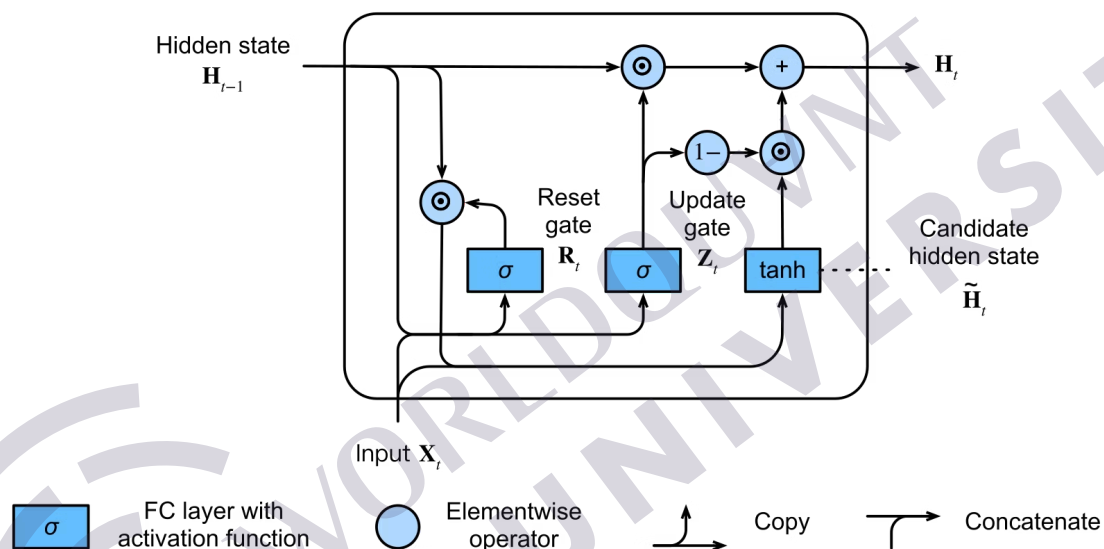
- the three input cells of the current time step,
- the output of itself from the previous time step (assuming there is a previous time step), and
- the output from the other hidden cell from the previous time step (again, if there is a previous time step).

1.2 Gated Recurrent Unit (GRU)

The gated recurrent unit (GRU) basically extends the RNN by adding two gates:

- The reset gate helps capture short-term dependencies in sequences by controlling how much of the previous state should be remembered.
- The update gate helps capture long-term dependencies in sequences because it reflects how much of the new state is just a copy of the old state (d2i 9.1).

Figure 2: A Single GRU Cell and Its Gates in the Hidden Layer



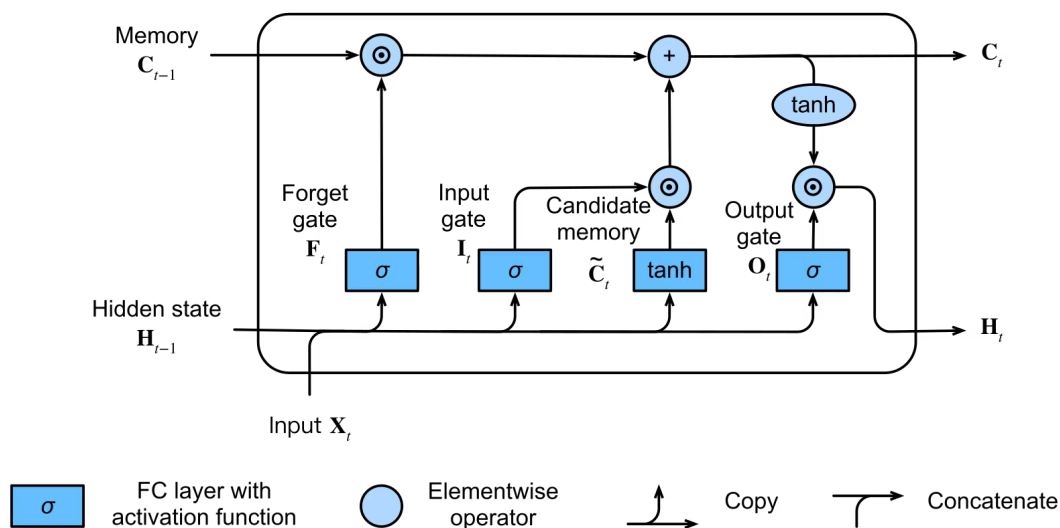
Source: Zhang, Aston et al. *Dive into Deep Learning*. https://d2l.ai/chapter_recurrent-modern/gru.html

As you can imagine (or recall from your machine learning course), the parameterization of these additional components equates to a higher level of complexity.

1.3 Long Short Term Memory (LSTM)

Finally, we arrive at the LSTM, which seems like a natural extension of the GRU in many ways although it was actually discovered much earlier. And just as the GRU is more complex than a simple RNN, the additional complexity of the LSTM also naturally results in longer training time.

Figure 3: LSTM Cell Architecture



Source: Zhang, Aston et al. *Dive into Deep Learning*. https://d2l.ai/chapter_recurrent-modern/lstm.html

The LSTM memory cell consists of four main components:

1. The input gate determines which information gets stored in the memory cell.
2. The forget gate, similar to the reset gate in the GRU, determines how much information from the previous state to include or exclude in the current processing.
3. The output gate combines the new input with the updated memory cell and passes this information to the next hidden layer.
4. The memory cell performs the function of long-term memory.

Lucky for us, the DeepAR module manages the LSTM details almost completely behind the scenes. While it is important to remind ourselves about the basic mechanics of this extremely popular and powerful instance of RNN, detailed knowledge of the component of each cell is not required for our productive experimentation with the DeepVaR model.

2. Backtesting VaR

2.1 Exceeding VaR vs. Exceeding Expectations

Since we will be discussing VaR techniques intensely throughout this and the next module, we need to consider ways of evaluating these techniques as well as the consequences of inaccurate results. Fatouros et al. discuss evaluation at some length in their paper, and the terminology can be a little confusing at times, so let's clarify some of their ideas and also put them in the context of other ideas about testing and backtesting VaR estimates.

It is worth noting that in "Section 3.2.3 Evaluation Metrics," VaR is assumed to be a negative number (Fatouros et al. 13–14). Therefore, in equation 7, the "Hit Variable" I_t takes the value 1 if $VaR_t > PnL_t$, meaning that the PnL—really, the loss—for day t was greater than the value-at-risk estimated loss threshold (Fatouros et al. 13). (The "Hit Variable" I_t takes the value 0 otherwise.) Again, this can be very confusing, so make sure you have completely adapted to the sign convention here: When the loss value associated with the VaR is exceeded, the result is a "Hit" or exceedance, which should happen about 5% of the time given a 95% VaR.

In fact, for a 100 days of PnL, we actually expect the PnL to be less than the VaR five times for 95% VaR or once for 99% VaR; we expect the loss to exceed the loss threshold that the VaR represents. Fatouros et al. refer to these numbers (5 and 1, respectively) as the "Expected Violations" (7), though this seems a bit like an oxymoron. These exceedances are expected; their number is a simple function of the confidence level for which we have calculated our VaR, so they are not really "violations." Fatouros et al. even define "Expected Violations" as "allowed exceedances" (7). Also, these exceedances certainly do not violate the assumptions of our model, for we would be completely and dangerously misunderstanding VaR if we assumed that losses would never exceed the threshold. Obviously, the authors do indeed understand VaR; it's just that their choice of words here could be misconstrued or misleading if we were not upfront here in an attempt to clarify.

2.2 Measures and Loss Functions for Backtesting VaR

To demonstrate, with Pythonic clarity, the metrics reviewed by Fatouros et al. (13-14), we take advantage of the script from the `backtest.py` script in Bayer's VaR-Backtesting repo. In it, he defines a `backtest` class with the parameters you might expect to be required:

- `actual`, an array of actual returns
- `forecast`, an array of VaR forecasts

- alpha, the confidence level for which we have calculated our VaR forecasts

For this class, we have the following methods, which are presented mainly to drive home the (albeit trivial) mathematics of the measures described by Fatouros et al. but also to present examples of elegantly written Python code:

```
def hit_series(self):

    return (self.actual < self.forecast) * 1
```

This succinct function assigns the appropriate value to the “Hit Variable” in equation 7 mentioned above (Fatouros et al. 13).

```
def hit_rate(self):

    return self.hit_series().mean()
```

This elegant function performs the calculation in equation 10, the so-called “violation rate” (Fatouros et al. 13).

```
def quadratic_loss(self, return_mean=True):

    loss = (self.hit_series() * (1 + (self.actual - self.forecast)**2))

    if return_mean:

        return loss.mean()

    else:

        return loss
```

Here, we have the “Quadratic Loss” from equation 11 (Fatouros et al. 13), with the option to calculate the mean instead of the sum. Note that the hit_series should be a binary array (check against the definition above), so that there is zero contribution to this loss when the loss is not as bad as the VaR threshold.

```
def smooth_loss(self, delta=25, return_mean=True):

    loss = ((self.alpha - (1 + np.exp(delta*(self.actual - self.forecast)))**(-1)) * (self.

    if return_mean:

        return loss.mean()

    else:

        return loss
```

Here we have the “Smooth Loss” from equation 12 (Fatouros et al. 13), again with the option to calculate the mean instead of the sum. The variable delta of course corresponds to the d in the Fatouros et al. reading and is parameterized with the same value, 25.

```
def tick_loss(self, return_mean=True):

    loss = (self.alpha - self.hit_series()) * (self.actual - self.forecast)

    if return_mean:

        return loss.mean()

    else:

        return loss
```

Here, we have the “Tick Loss” from equation 13 (Fatouros et al. 14), again with the option to calculate the mean instead of the sum.

```
def firm_loss(self, c=1, return_mean=True):

    loss = (self.hit_series() * (1 + (self.actual - self.forecast)**2) - c*(1-self.hit_se

    if return_mean:

        return loss.mean()

    else:

        return loss
```

Here, we have the “Firm Loss” from equation 14 (Fatouros et al. 14), again with the option to calculate the mean instead of the sum. Note that Fatouros et al. use the variable a to capture “the opportunity cost of capital” and set it equal to one (14); in the code presented here, the variable is c but still set to one.

2.3 Validity Tests for Backtesting VaR

There are two tests mentioned by Fatouros et al., which go beyond merely measuring and penalizing inaccurate VaR estimates and unexpected exceedances (14). They actually test the validity of certain fundamental assumptions of the Value at Risk models. We follow Bayer’s naming conventions for these in the discussion below.

Christoffersen’s “Likelihood Ratio test,” which as you will see is actually a set of two tests, checks the assumptions of unconditional coverage and independence.

1. The unconditional coverage hypothesis is that the probability of VaR exceedance is in fact $1 - \alpha$, where α is the confidence level for which we have estimated our VaR. If the actual exceedance rate is higher or lower than $1 - \alpha$, then the model underestimates or overestimates the true level of risk.
2. The independence hypothesis asserts that VaR exceedances must be distributed independently. Whether an exceedance occurred on day t should not provide any information about whether an exceedance occurs on day $t + 1$. There should be no “clustering” of exceedances (even if the overall number or rate of exceedances still approximates the expected $1 - \alpha$ (Dumitrescu 83–84).

See and work through Bayer’s function `lr_bt` (for likelihood ratio backtest) for a straightforward implementation and a better understanding. Bayer’s entire `backtest.py` file is a required reading.

Engle and Managanelli’s “Dynamic Quantile test” also tests for independence but using a different approach: First they fit a linear regression model to the historical exceedances (Manchester 14). If the null hypothesis that all the coefficients in the linear regression equation are in fact zero is not rejected using a chi-square test, then newer exceedances are indeed independent of the previous ones.

Again, see and work through Bayer's function `dq_bt`, which does all of this quite elegantly.

3. Conclusion

In this lesson, we learned about a cutting-edge model that combines Monte Carlo simulation techniques with an LSTM architecture that provide probabilistic forecasts from which we can estimate VaR. We also reviewed LSTMs in relation to RNNs. In the next lesson, DeepVaR comes alive with a Python implementation that shows how easily this sophisticated technology can be put to use right now with actual financial data.

References

- Bayer, Sebastian. "VaR-Backtesting." *GitHub*, <https://github.com/BayerSe/VaR-Backtesting>.
- Dumitrescu, Elena-Ivona, et al. "Backtesting Value-at-Risk: From Dynamic Quantile to Dynamic Binary Tests." *Finance*, vol. 33, 2012, pp. 79–112, <https://www.cairn.info/revue-finance-2012-1-page-79.htm?contenu=article>.
- Fatouros, Georgios, et al. "DeepVaR: A Framework for Portfolio Risk Assessment Leveraging Probabilistic Deep Neural Networks." *Digital Finance*, 13 April 2022, <https://link.springer.com/article/10.1007/s42521-022-00050-0>.
- giorgosfatouros. "DeepVaR." *GitHub*, <https://github.com/giorgosfatouros/DeepVaR>.
- Xie, Chenlei et al. "A Long Short-Term Memory Neural Network Model for Knee Joint Acceleration Estimation Using Mechanomyography Signals." *International Journal of Advanced Robotic Systems*, vol. 17, no. 6, 2020, pp. 1–10. https://www.researchgate.net/publication/346666554_A_long_short-term_memory_neural_network_model_for_knee_joint_acceleration_estimation_using_mechanomyography_signals
- Zhang, Aston et al. *Dive into Deep Learning*. <https://d2l.ai/index.html#>
- Zhang, Y, and S. Nadarajah. "A Review of Backtesting for Value at Risk." *Communications in Statistics - Theory and Methods*, 2017, pp. 1–24. <https://www.research.manchester.ac.uk/portal/files/60673220/back4.pdf>